PART A

OBJECTS, REFERENCES MEMORY

YOUR LEARNING OBJECTIVES

- new creates an object; variables hold reference values.
- Assignment copies a reference; it does not duplicate the object.
- Java passes all arguments by value, even when the values are references.
- == tests reference identity; equals() may test logical equality.
- final does not freeze mutable objects; GC is based on reachability.

19 focused concept lessons • one infographic

Mini exercises and reasoned answers included

Understand identity, aliasing, pass-by-value, equality, mutability and reachability.

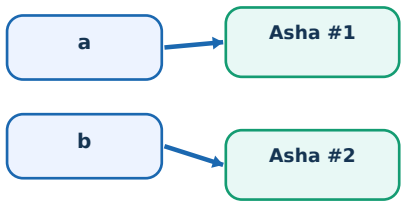
Objects are instances; classes describe them

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A class defines a type. new creates a distinct object of that type, invokes a constructor, and produces a reference that you can store. Identical field values do not imply identical objects. a and b receive separate references because new executed twice. A reference variable is not the object itself. Java checks allowed operations from the declared type.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("Asha");
Person b = new Person("Asha");
System.out.println(a == b);
System.out.println(a.getName());
```

EXPECTED OUTPUT

```
false
Asha
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The first new runs a constructor and returns a reference to object #1.
2. The second new returns a reference to object #2, even though its name is also Asha.
3. == compares those references; getName() reads a field on object #1.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A variable declaration such as Person p; only reserves a reference slot. Construction is a separate expression. The declared type controls what members the compiler lets you call; the object's identity and its field values are different properties. A failed constructor does not return a successfully constructed object.

COMMON TRAP

Do not equate a variable name with a physical heap address; references are abstract language-level values.

TRY THIS IN INTELLIJ

Replace b with a. Predict the equality check.

REMEMBER Two new expressions normally mean two distinct objects.

APPLY THE RULE / CHANGE ONE LINE

Replace the second new Person("Asha") with Person b = a;

Now a == b is true: the same object has two aliases rather than two identities.

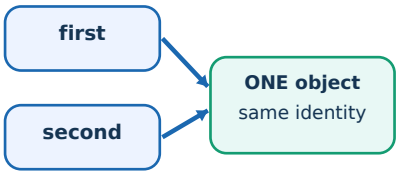
A practical stack-versus-heap mental model

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Think of local variables as slots in the running method and objects as separately allocated entities; Java does not promise a particular physical location for every value. Both local variables hold the same reference value; there is one Person object in this example. The JVM may optimize allocation, so diagrams of “stack references / heap objects” are conceptual teaching models, not guaranteed memory layouts.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person first = new Person("Asha");
Person second = first;
System.out.println(first == second);
```

EXPECTED OUTPUT

```
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. first stores a reference returned by the constructor.
2. second = first copies that same reference value into a second local variable.
3. == is true because both variables identify the same object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A stack/heap picture is a reasoning aid: method invocations have local variables and objects have independent identity. JVM optimizations can keep values in registers, inline data, or remove allocations. None of those optimizations changes the Java-visible result: two aliases can refer to one object.

COMMON TRAP

A String or an object field can hold references too; not every reference is in a stack frame.

TRY THIS IN INTELLIJ

Draw the two variable arrows pointing to a single box.

REMEMBER Draw reference arrows, not physical memory addresses.

APPLY THE RULE / COMPARE

Replace second = first with second = new Person("Asha");

The name values match, but the references now identify separate objects; == is false.

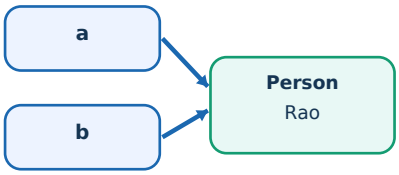
Reference assignment creates an alias

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Assigning one reference variable to another copies the reference value; subsequent mutations through either alias affect the same object. There are two variables but one Person. The method call changes the state of the object that both variables identify. Reassigning b later does not change a: aliases are copied values, not linked variables.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("Asha");
Person b = a;
b.setName("Rao");
System.out.println(a.getName());
System.out.println(a == b);
```

EXPECTED OUTPUT

```
Rao
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new creates one mutable Person, initially named Asha.
2. b = a copies the reference; it does not create another Person.
3. b.setName updates the shared object, so a.getName() reads Rao.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Aliasing matters any time a method holds a reference received from elsewhere. If either alias mutates shared state, all observers see that state when they read it. By contrast, b = new Person(...) changes only b's reference slot. Copying a reference is cheap, but sharing mutability needs deliberate ownership.

COMMON TRAP

Confusing assignment of a reference with cloning an object is a frequent bug.

TRY THIS IN INTELLIJ

Now set b = new Person("New"); and print a.getName().

REMEMBER Reference assignment copies an arrow, not an object.

APPLY THE RULE / MUTATION VS REBINDING

After the setter, add b = new Person("New");

a still reads Rao. Rebinding b changed no field on the original Person.

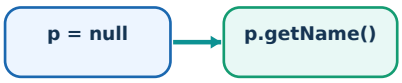
null: a reference with no object

Understand the rule • trace the code • read the diagram • test it

CORE RULE

null is a special value that can be assigned to reference types. It does not point to an object, so dereferencing it is invalid. The equality test is safe; invoking an instance method on null throws NullPointerException. A field may default to null; local reference variables must be definitely assigned before use.

VISUAL MODEL



NO TARGET
NPE

CODE EXAMPLE / ALLMAN BRACES

```
Person p = null;
System.out.println(p == null);
try
{
    System.out.println(p.getName());
}
catch (NullPointerException ex)
{
    System.out.println("Cannot dereference null");
}
```

EXPECTED OUTPUT

```
true
Cannot dereference null
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. p holds null, not a reference to an existing Person.
2. p == null safely evaluates to true without dereferencing anything.
3. p.getName() attempts an instance call through null and throws NullPointerException, which the catch handles.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Distinguish a null reference from an uninitialized local variable. Instance fields default to null, but local references require definite assignment. Prefer clear non-null APIs and validation at boundaries; Objects.requireNonNull can fail early with a helpful message. Catching NPE is usually not a substitute for correct design.

COMMON TRAP

A null check just before use is not a universal concurrency guarantee; prefer clear ownership and non-null invariants.

TRY THIS IN INTELLIJ

Use if (p != null) to guard a call and predict what prints.

REMEMBER Checking null is safe; dereferencing null is not.

ONE MORE QUESTION / SAFER ALTERNATIVE

Guard the call with if (p != null) before using p.getName().

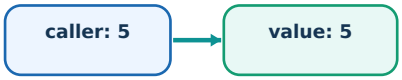
Pass-by-value with primitive parameters

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Java always passes argument values to parameters. For primitives, the parameter gets a copy of the numeric or boolean value. The parameter named value is local to the called method. Incrementing it does not write back to count. A return value can communicate the computed result. Calling count = addOne(count) would require addOne to return int.

VISUAL MODEL



COPY
caller stays 5

CODE EXAMPLE / ALLMAN BRACES

```
static void addOne(int value)
{
    value++;
}
int count = 5;
addOne(count);
System.out.println(count);
```

EXPECTED OUTPUT

```
5
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. count contains the primitive int value 5 in the caller.
2. The call copies that value into the method's separate parameter slot named value.
3. value++ changes only the parameter; the caller's count remains 5.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Java has no ordinary pass-by-reference parameters. A method can compute a new value and return it, but the caller must assign the result. The callee receives the evaluated argument expression, not access to the caller's local variable slot. That remains true if you call addOne(count + 10).

COMMON TRAP

Java has no pass-by-reference parameter syntax for ordinary methods.

TRY THIS IN INTELLIJ

Change addOne to return value + 1 and assign the result.

REMEMBER Primitive parameters receive copied values.

APPLY THE RULE / RETURN A RESULT

Change the method to return value + 1 and write count = addOne(count);

The caller explicitly stores the returned result; count then becomes 6.

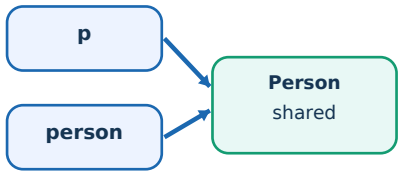
Pass-by-value with object references

Understand the rule • trace the code • read the diagram • test it

CORE RULE

For objects, Java passes a copy of the reference value. The called method can use it to reach and mutate the same object. The reference was copied, but both copies identify the same Person instance during the call. This is not a special pass-by-reference rule; the copied value simply happens to be a reference.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static void rename(Person person)
{
    person.setName("New name");
}
Person p = new Person("Old name");
rename(p);
System.out.println(p.getName());
```

EXPECTED OUTPUT

```
New name
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. p receives a reference to the Person named Old name.
2. rename(p) copies that reference into the method parameter person.
3. person.setName updates the one shared Person, so the caller observes New name.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The copies are references to the same object, not copies of the object itself. Think in two layers: parameter assignments affect local slots, while setter calls affect object state. This distinction works the same with List, Map, and interface-typed parameters. Passing null succeeds as an argument but dereferencing it inside rename would fail.

COMMON TRAP

“Objects are passed by reference” is a misleading shortcut in Java discussions.

TRY THIS IN INTELLIJ

Would rename(null) fail at call time or at the dereference?

REMEMBER Pass-by-value can still produce visible object mutations.

APPLY THE RULE / FAILURE LOCATION

Call rename(null) with the original method body.

Passing null is permitted. The exception occurs when person.setName dereferences it.

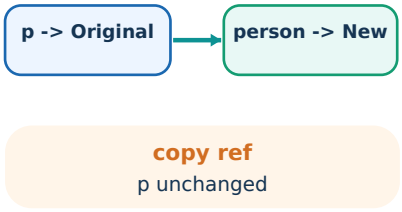
Rebinding a parameter does not rebind the caller

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Reassigning an object parameter replaces only the local parameter reference. It does not replace the reference stored in the calling method. Java first copies p into person. Assigning a new reference to person cannot alter p. Contrast this with person.setName(...): that operation changes the shared object rather than the parameter slot.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static void replace(Person person)
{
    person = new Person("Replacement");
}
Person p = new Person("Original");
replace(p);
System.out.println(p.getName());
```

EXPECTED OUTPUT

```
Original
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The caller's p and the parameter person initially identify the Original object.
2. person = new Person(...) creates object #2 and changes only the parameter slot.
3. After the method returns, p still identifies object #1, so Original prints.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

You can mutate an object through a parameter and later rebind the parameter; the earlier mutation persists. What never happens is Java silently writing a new reference back into the caller's variable. To replace the caller's object, return a new reference and explicitly assign it: p = replace(p).

COMMON TRAP

Two methods that “receive a Person” may behave very differently: one mutates, the other merely rebinds.

TRY THIS IN INTELLIJ

Combine replace with a mutation before reassignment; what changes persist?

REMEMBER Rebinding a parameter cannot redirect a caller's variable.

APPLY THE RULE / MUTATE THEN REBIND

Call person.setName("Changed") before person = new Person(...).

p still refers to the original object, but its name becomes Changed.

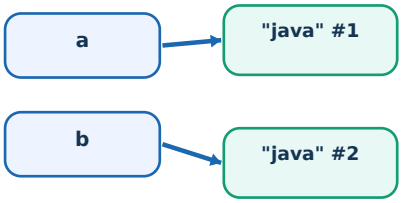
Identity comparison: == for references

Understand the rule • trace the code • read the diagram • test it

CORE RULE

For two reference expressions, == asks whether they refer to the same object (or are both null), not whether their contents match. This deliberate use of new constructs distinct String instances, while equals compares their character sequences. The behavior of == for primitive expressions differs: it compares primitive values after applicable numeric promotion.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);
System.out.println(a.equals(b));
```

EXPECTED OUTPUT

```
false
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new String creates a fresh String object for a; the second new creates another for b.
2. a == b is false because the two references identify different objects.
3. a.equals(b) is true because both strings contain the characters java.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

String literals may refer to pooled objects, so comparing two identical literals with == may sometimes look correct. It is still the wrong question when you mean textual equality. Use a.equals(b) when a is non-null or Objects.equals(a, b) when either side can be null. Identity tests are valid only when sameness is intentional.

COMMON TRAP

String interning may make == appear to work for some literals. Never use that as a text comparison strategy.

TRY THIS IN INTELLIJ

Test String a = "java"; String b = "java"; then explain why that case is different.

REMEMBER For text content, use equals(), not ==.

APPLY THE RULE / WHY LITERALS MISLEAD

Try String a = "java", b = "java"; with ==.

Interned literals can compare true by identity; that does not make == a text comparison.

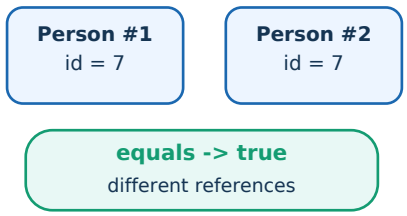
equals(): logical equality and contracts

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Classes may define logical equality by overriding equals(Object). The contract includes reflexive, symmetric, transitive, consistent, and null-safe behavior. This assumes Person.equals is based on stable id, as in the companion project. Different instances can then be logically equal. An equality definition should use fields that match the domain meaning of identity and should remain stable when objects are used as keys.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person(7, "Asha");
Person b = new Person(7, "Rao");
System.out.println(a == b);
System.out.println(a.equals(b));
```

EXPECTED OUTPUT

```
false
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The two Person constructors return distinct references, making == false.
2. The project Person.equals implementation compares a stable id instead of the name.
3. Both objects have id 7, so equals returns true even though their names differ.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An equals override must accept Object and respect reflexivity, symmetry, transitivity, consistency, and false for null. Decide whether your type represents entity identity or a set of values. Fields used in equality should ideally be stable while the object is in collections. An equals(Person) overload does not override Object.equals.

COMMON TRAP

Overloading equals(Person) instead of overriding equals(Object) is not the standard equality override.

TRY THIS IN INTELLIJ

Check a.equals(null), b.equals(a), and a.equals(a).

REMEMBER Logical equality is a class-defined contract, not reference identity.

APPLY THE RULE / OVERRIDE VS OVERLOAD

Write equals(Person p) but leave equals(Object) unchanged.

Collections and Object-typed calls still use the inherited Object.equals implementation.

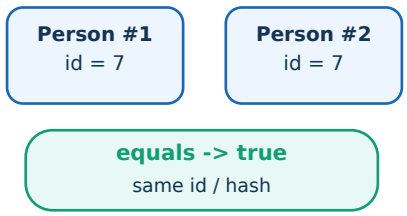
hashCode must agree with equals

Understand the rule • trace the code • read the diagram • test it

CORE RULE

If two objects are equal under equals, they must have the same hashCode. The reverse is not required: collisions are allowed. Collections that use hashing rely on this contract. Violation can make an equal object impossible to find in a hashed structure. Java Object supplies default identity-based implementations; override both when defining value equality.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person(7, "Asha");
Person b = new Person(7, "Rao");
System.out.println(a.equals(b));
System.out.println(a.hashCode() == b.hashCode());
```

EXPECTED OUTPUT

```
true
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The objects use the same equality-defining id, so equals returns true.
2. Their hashCode implementations use that same id, so hash codes match.
3. A hash-based collection can use the consistent hash to locate a candidate bucket before checking equals.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The one-way rule is: equal objects MUST have equal hash codes. Unequal objects can collide. A hash code is not a unique identifier or an address. If a key changes its hash-relevant fields after insertion into HashMap or HashSet, lookups may fail even though the object still exists.

COMMON TRAP

Never use mutable fields in equality/hashCode for objects that remain in hashed collections unless you control mutation carefully.

TRY THIS IN INTELLIJ

What could happen if a key's hash-relevant field changes after insertion?

REMEMBER When overriding equals(), also override hashCode().

APPLY THE RULE / HASH COLLECTION WARNING

Insert an object into a HashSet, then change a field used by hashCode.

The set may no longer locate the object using the changed hash value.

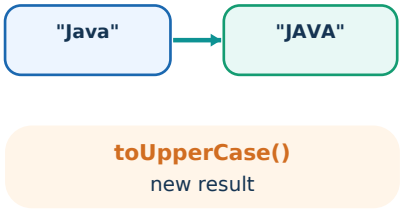
String immutability is different from variable immutability

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A String cannot change its character sequence. A String variable can still be rebound to a different String unless the variable is final. toUpperCase returns a String value; ignoring it leaves text unchanged. The second assignment updates the variable to a new result. Immutable objects are easier to share safely because callers cannot change their internal state.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
String text = "Java";
text.toUpperCase();
System.out.println(text);
text = text.toUpperCase();
System.out.println(text);
```

EXPECTED OUTPUT

```
Java
JAVA
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. text initially refers to the immutable String Java.
2. toUpperCase returns a result, but the first return value is ignored; text still prints Java.
3. The assignment text = text.toUpperCase() rebinds the variable; the next print is JAVA.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Immutability describes an object's state, not whether a variable can change its reference. A String method such as substring, strip, replace, or concat returns a value rather than changing the receiver. For lots of character-by-character edits, StringBuilder provides a mutable buffer, but it is a different type.

COMMON TRAP

Do not assume every method whose name sounds like an edit mutates its receiver.

TRY THIS IN INTELLIJ

Predict text.concat(" 17") without assigning the result.

REMEMBER An immutable object can be referenced by a reassignable variable.

APPLY THE RULE / IGNORED RETURN VALUE

Run text.concat(" 17"); without assigning its result.

The original String is unchanged. Print text to verify before and after.

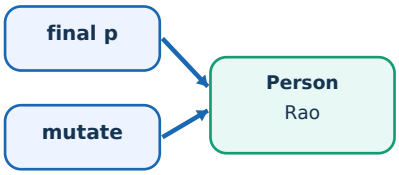
final reference versus mutable referent

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A final local reference cannot be reassigned after initialization, but that restriction does not freeze the referenced object. final applies to the variable slot. The mutable Person instance still permits valid changes through its methods. For deeper immutability, design the object with private final fields and do not expose mutable internal objects.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
final Person person = new Person("Asha");
person.setName("Rao");
System.out.println(person.getName());
// person = new Person("Other"); // Error
```

EXPECTED OUTPUT

Rao

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. final binds the local variable person once to a Person reference.
2. person.setName mutates the Person, so the new name Rao is visible.
3. Reassigning person is a compile-time error, even though mutating its fields was allowed.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

final applies at the variable-reference layer. Immutable object design goes further: private final fields, no mutators, and no leaking mutable members. A final List variable can still call add() on a mutable list. Keep the distinction between 'cannot rebind' and 'cannot change observable state'.

COMMON TRAP

Declaring List or Map references final does not make their contents immutable.

TRY THIS IN INTELLIJ

Design an immutable Name object with one final field and no setter.

REMEMBER final reference does not mean immutable object.

APPLY THE RULE / FINAL IS SHALLOW

Make a final reference to a mutable ArrayList, then call add().

The list contents change normally even though the reference cannot be rebound.

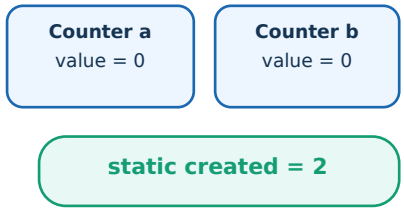
Fields, local variables, and static state

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Instance fields belong to each object. Static fields belong to the class. Local variables belong to a method invocation and require definite assignment. Each new Counter has its own value field, initially zero; created is shared across all Counter instances. Field default initialization occurs before constructor logic. A local int x; does not receive a usable default.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Counter
{
    static int created;
    int value;
    Counter()
    {
        created++;
    }
}
Counter a = new Counter();
Counter b = new Counter();
System.out.println(Counter.created);
```

EXPECTED OUTPUT

```
2
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Constructing a initializes its own instance value field and increments Counter.created.
2. Constructing b gets a different instance field but increments the same static created field.
3. Counter.created prints 2 because the static counter is associated with the class.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Fields receive default values before constructor code executes: numeric zero, false, or null. A local variable has no usable default; Java's definite-assignment analysis prevents reading it uninitialized. static changes ownership, not visibility: static fields still follow their access modifiers, and shared mutable static data can be a concurrency risk.

COMMON TRAP

Shared mutable static state can create surprising interactions and thread-safety concerns.

TRY THIS IN INTELLIJ

Set a.value=9. Predict b.value.

REMEMBER Instance fields are per object; static fields are shared per class.

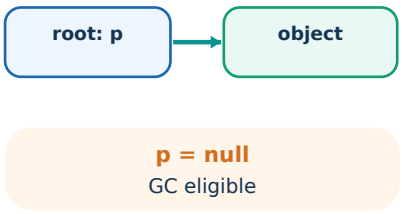
Garbage collection and object reachability

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Objects become eligible for garbage collection when they are no longer reachable through live references. Collection timing is not specified. Clearing the only live reference may make an object eligible for collection, but JVM optimization and other references affect reachability. The JVM handles memory reclamation; GC does not replace deterministic closing of files, sockets, or database connections.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person p = new Person("Short lived");
p = null;
System.out.println("Reference cleared");
```

EXPECTED OUTPUT

```
Reference cleared
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new makes a Person object reachable through the local p.
2. p = null removes that local path to the object.
3. The print statement only confirms that the reference was cleared; it does not prove GC ran.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Reachability is what matters to garbage collection, not the execution of a particular null assignment. The runtime may delay collection and optimize object lifetime. Do not build behavior around finalizers or System.gc(). File handles, sockets and database connections have separate lifecycles; use try-with-resources for reliable closing.

COMMON TRAP

Do not predict exactly when finalize or garbage collection will run, and do not use System.gc() as a correctness mechanism.

TRY THIS IN INTELLIJ

If another variable holds the same reference, is the object eligible?

REMEMBER Eligible for GC is not the same as collected now.

APPLY THE RULE / GC IS NOT A CALLBACK

Call System.gc() and then print a status message.

Only the message is deterministic; collection timing is not guaranteed.

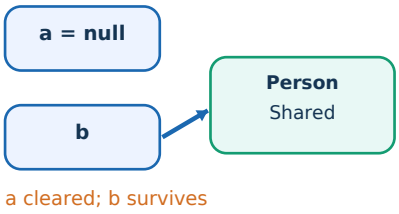
Reachability versus alias count

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An object can remain reachable through instance fields, static fields, locals, or other objects even if one variable is cleared. b still reaches the object, so clearing a does not make that shared object unreachable. Real object graphs can have cycles; Java garbage collectors use reachability rather than simple per-object reference counting.

VISUAL MODEL



a cleared; b survives

CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("Shared");
Person b = a;
a = null;
System.out.println(b.getName());
```

EXPECTED OUTPUT

```
Shared
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. a receives a reference to the new Person Shared.
2. b = a makes b a second path to the same Person.
3. a = null removes one path, but b still reaches the object and prints Shared.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

GC traces paths from live roots, through object fields and collections, to reachable objects. A cycle can still be reclaimed if no roots reach it. A static cache or long-lived collection can unintentionally hold objects alive. Ask 'is there any path from a live root?' rather than counting how many pointers were assigned.

COMMON TRAP

A cycle alone does not keep objects alive if nothing reachable points into that cycle.

TRY THIS IN INTELLIJ

Draw the arrows before and after a=null.

REMEMBER One surviving path is enough to keep an object reachable.

APPLY THE RULE / TRACE THE LAST PATH

After a = null, also set b = null (assuming no other roots).

The object may become unreachable; that still does not say when reclamation occurs.

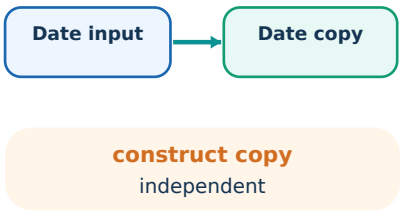
Defensive copying prevents ownership leaks

Understand the rule • trace the code • read the diagram • test it

CORE RULE

If a class accepts or returns a mutable object, it may need a copy to preserve its invariants and avoid external modifications. Date is mutable, so copying the timestamp separates ownership. java.time immutable classes are usually preferable for new code. Create copies both on input and output when a class must shield mutable fields from callers.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Date input = new Date(0);
Date copy = new Date(input.getTime());
input.setTime(99_000);
System.out.println(copy.getTime());
```

EXPECTED OUTPUT

```
0
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. input is a mutable Date representing 0 milliseconds since the epoch.
2. new Date(input.getTime()) constructs a separate Date with the same timestamp.
3. Changing input to 99,000 does not change copy, so copy prints 0.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

For a class that stores mutable caller-owned data, copy on input so the caller cannot change your internals later. Copy on output so readers cannot mutate your internals through a getter. A shallow copy of a container may still share nested mutable elements. Prefer java.time immutable date/time values for new APIs.

COMMON TRAP

Copying just the reference does not create a defensive copy. A shallow copy may still share nested mutable objects.

TRY THIS IN INTELLIJ

What happens if copy=input instead of new Date(...)?

REMEMBER Defensive copying separates ownership only as deeply as you copy.

APPLY THE RULE / NO COPY AT ALL

Replace the constructor call with Date copy = input; and mutate input.

Now copy and input alias one Date, so copy.getTime() becomes 99000.

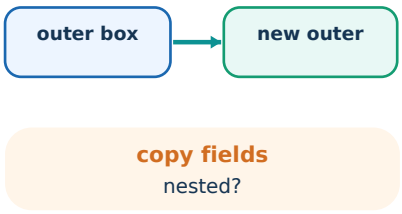
Copy constructors, shallow and deep copies

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A copy constructor can create a new outer object. Whether nested data is copied determines if the result is shallow or deep. For a primitive field, copying its value is straightforward. For a mutable nested reference, you must decide whether to create an independent nested object. Cloneable and Object.clone exist, but explicit constructors or factories are often clearer about what gets copied.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Box
{
    int value;
    Box(int value)
    {
        this.value = value;
    }
    Box(Box other)
    {
        this.value = other.value;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. A copy constructor receives an existing Box as input.
2. It constructs a new outer Box and copies the primitive value into its own field.
3. If Box later owns a mutable Date or List, merely copying that reference would still share nested state.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

'Deep copy' depends on the shape of the object graph and your ownership contract. Immutable nested values can safely be shared. Mutable nested objects may require new independent instances. Cycles and duplicated references complicate naive recursive copying; explicit constructors/factories often make copying intent clearer than clone().

COMMON TRAP

A “copy” method is not necessarily deep: inspect every mutable field.

TRY THIS IN INTELLIJ

Add a mutable Date field. How would you copy it defensively?

REMEMBER New outer object does not automatically mean deep copy.

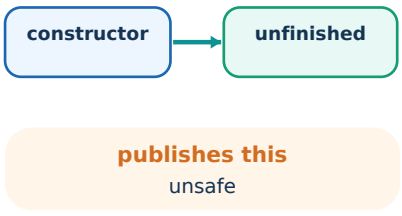
Object escape during construction

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Publishing this before a constructor finishes can expose a partially initialized object to other code or threads. Normally keep this private until initialization finishes. Constructor injection and factories can help control publication. Overridable method calls inside constructors are also risky: they may dispatch to a subclass before its fields initialize.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Unsafe
{
    static Unsafe published;
    String name;
    Unsafe()
    {
        published = this; // avoid early escape
        name = "ready";
    }
}
```

EXPECTED OUTPUT

```
No console output (illustrates design risk).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The Unsafe constructor begins before the name field is assigned ready.
2. published = this exposes a reference to the in-progress object.
3. Only afterward does the constructor set name; other observers may have seen incomplete state.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Constructor escape is particularly dangerous when a listener, callback, static registry, or other thread can inspect the object before setup is complete. Calling overridable methods from a superclass constructor can invoke subclass code before subclass initialization. Finish validation and initialization, then publish the fully built object through a factory or normal return path.

COMMON TRAP

The danger may be subtle, especially with concurrent execution or inheritance.

TRY THIS IN INTELLIJ

Why is calling an overridable method from a base constructor hazardous?

REMEMBER Construct completely first; publish the reference afterward.

ONE MORE QUESTION / SAFE CONSTRUCTION RULE

Move publication to a factory after the constructor returns.

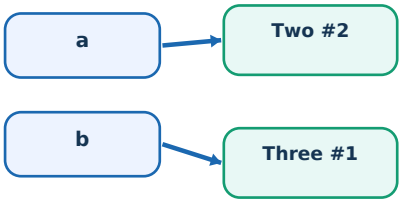
Reference tracing: one object or two?

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Read each assignment as moving reference values between variable slots. Mutations change objects; reassignment changes only the selected slot. The first new creates object #1. b aliases it. The second new creates object #2, and only a is rebound. b continues to refer to object #1. When debugging state changes, draw arrows to objects rather than mentally treating variables as containers of entire objects.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person a = new Person("One");
Person b = a;
a = new Person("Two");
b.setName("Three");
System.out.println(a.getName());
System.out.println(b.getName());
System.out.println(a == b);
```

EXPECTED OUTPUT

```
Two
Three
false
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The first new makes object #1 named One, and b aliases a.
2. The second new makes object #2 named Two; only a is rebound to object #2.
3. b.setName changes object #1 to Three, so prints are Two, Three, false.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A reliable trace uses one box per new expression, one arrow per reference variable, and state updates only for setters. At each assignment, erase/redraw just the variable's arrow. This works for larger object graphs too, including services with shared dependencies and collections that store object references.

COMMON TRAP

Do not confuse two variable names with two objects, or one variable name with only one lifetime object.

TRY THIS IN INTELLIJ

Change the order of the last three statements and predict the output.

REMEMBER To trace state, count objects and track arrows separately.

APPLY THE RULE / ARROWS FIRST

After the second new, ask which object b points to before changing its name.

b still points to object #1; a points to object #2. The setter affects only #1.

CHAPTER 3 · PART A · VISUAL MASTER MAP

Core rules • mental models • common pitfalls • executable examples



01 OBJECT CREATION

- new constructs a distinct object.
- A variable stores a reference, not a whole object.
- The declared type controls callable members.

02 ALIASING

- b = a copies a reference value.
- Both variables may identify one object.
- Mutate object vs rebind variable.

03 PASS-BY-VALUE

- Every parameter gets a copy.
- Reference copies can mutate shared objects.
- Rebinding a parameter cannot rebind caller.

04 IDENTITY & EQUALITY

- == checks same reference for objects.
- equals can define logical equality.
- Equal objects require equal hash codes.

05 MUTABILITY

- String is immutable.
- final reference prevents rebinding.
- Defensive copies protect mutable state.

06 MEMORY & GC

- Stack/heap diagrams are conceptual.
- GC eligibility follows reachability.
- Use explicit close for external resources.

EXAMPLE — SOURCE AND OUTPUT

CODE / RULE

```
Person a = new Person("Asha");
Person b = a;
b.setName("Rao");
System.out.println(a.getName());
```

EXPECTED RESULT

Rao

PART A / PRACTICE

Predict before running

9 thinking questions. Answer from the Java rule, not from memorized output.

A1 Two variables alias one Person. Which operation changes both views: b.setName(...) or b = new Person(...)?

A2 Explain why a primitive parameter increment cannot update the caller local.

A3 Can a callee replace a caller’s Person variable by reassigning its parameter?

A4 Two new Person objects have the same ID. Predict == and equals under ID equality.

A5 If equals returns true for a and b, what must hashCode guarantee?

A6 Does final Person p mean p cannot be mutated?

A7 After a = null but b still references the object, can it be collected?

A8 Why is returning an internal java.util.Date risky?

A9 Why is calling an overridable method from a constructor risky?

Reasoning and answers

Explain each rule in your own words before moving on.

A1 Mutation of the shared object through b changes what a sees; rebinding b does not change a.

A2 The parameter holds a copy of the primitive value. Return the new value and assign it in the caller.

A3 No. A parameter gets a copied reference value; local reassignment cannot rebind the caller slot.

A4 == is false for two distinct objects; ID-based equals is true.

A5 a.hashCode() must equal b.hashCode(); equal hash codes alone do not imply equality.

A6 No. final prevents reassignment of p; allowed methods may still mutate the Person.

A7 Not while b remains a live reachability path to the object.

A8 Date is mutable; clients could modify internal state through the returned reference.

A9 Runtime dispatch may reach subclass code before subclass initialization is complete.